

Does Relational Information Improve Stock Return Ranking?

A Replication of Temporal Relational Ranking for Stock Prediction

Abstract

Write last, once Sections 5–7 have final numbers.

1 Introduction

A stock’s future return is usually modeled as a function of its own history: its own past prices, its own past volumes, its own fundamentals. This is convenient, but it discards information that is easy to observe and often economically meaningful — companies in the same industry, connected by a supply chain, or held together in the same funds tend to move together, because a shock to one is frequently a shock to the group. A model that treats every stock as an independent time series is, by construction, blind to that channel.

Feng et al. (2019) address this directly. Their Relational Stock Ranking (RSR) model encodes a graph of relations between stocks — who shares an industry, who is connected through Wikidata facts about ownership, supply, and competition — into a Temporal Graph Convolution layer, and combines it with a sequential (LSTM-based) encoding of each stock’s own price history. The relational signal is used to re-weight and propagate information between connected stocks at every training step, and the model is trained to rank stocks by next-day return rather than to regress the return value directly, since for a long-short or top- k trading strategy the relative ranking of stocks matters more than the exact predicted return.

This project replicates RSR on the same NASDAQ universe and industry-relation graph used in the original paper, asking whether its central empirical claim — that encoding relational information improves next-day return ranking relative to a relation-blind baseline — holds up when the model is retrained from scratch on the authors’ own released data and code.

Our interest in this specific question comes from outside this course: we separately maintain a multi-agent system for trading and investment decisions, in which individual agents each analyze a single stock along a different dimension (technical signals, news/sentiment, risk) before their outputs are aggregated into a decision. That system currently has no agent whose input is the relationship between stocks — every agent’s view of a stock is, in effect, independent of every other stock, the same simplifying assumption Feng et al. (2019) argue against. RSR is a concrete, reproducible instance of turning “stocks are related” into a model input (a relation graph plus a learned attention weight over neighbors) rather than an informal claim, which makes it a useful first test of whether that kind of signal is worth adding to such a system. This project does not attempt that integration or extend RSR’s architecture; it replicates RSR on its own terms, as a first check of whether the relational signal it relies on holds up before any downstream use is considered.

1.1 Research question

- **RQ1 (replication).** Does encoding a relation graph between stocks, following Feng et al. (2019), improve next-day return ranking relative to a baseline that treats stocks independently?

1.2 Contribution

We replicate the RSR baseline and relational model on the official NASDAQ dataset, training both to convergence under a matched schedule. RQ1 does not hold in the direction the original paper reports: at convergence, encoding the static industry relation graph does not lower mean-squared prediction error relative to the no-relation baseline, and the two models split on the two ranking-adjacent metrics — the baseline has the higher top-1 ranking accuracy (mrrt), while the relational model has the higher simulated trading return (btl). We report this split result, and the environment and data issues we had to resolve to obtain it (Sections 3 and 4), as the project’s substantive findings.

2 Related Work

Relation-based stock prediction. Feng et al. (2019) is the paper this project replicates; Section 4 describes its architecture in the detail needed to reproduce it. Cui et al. (2023) extend the same broad idea — that a stock’s neighbors carry predictive information — from pairwise relations to group-wise hypergraph relations (industry-belonging and fund-holding hyperedges), with a hierarchical attention mechanism over intra-hyperedge, inter-hyperedge, and inter-hypergraph structure. Jafari and Haratizadeh (2022) instead use a graph convolutional network directly on a stock-relation graph for price-movement classification; the authors note in their code release that the implementation was withheld pending journal review, so we treat this as a design reference rather than a reproducible baseline. All three approaches share the same core assumption this project tests directly: that encoding a graph of relations between stocks improves prediction relative to treating each stock independently.

3 Data

3.1 Source

We use the NASDAQ portion of the dataset released with Feng et al. (2019) (`Temporal_Relational_Stock_Ranking` official repository), which contains: (i) daily end-of-day features for 1,026 NASDAQ tickers — 5-, 10-, 20-, and 30-day moving averages of close price plus the raw close-to-close return ratio, already normalized by the stock’s maximum price over the sample — and (ii) a static industry-relation graph over the same 1,026 tickers, a one-hot encoding of shared-industry membership across 97 industry categories sourced from sector/industry classifications.

The repository’s own documentation describes the underlying raw pull as “historical (30 years) End-of-day data . . . of more than 8,000 stocks.” That describes the unfiltered Google Finance pull, not the modeling dataset actually used for training: the processed feature files cover **1,246 trading days per stock, roughly five years**, not thirty, and only 1,026 of the raw universe’s stocks survive the paper’s own liquidity/history filtering. A reader relying on the top-level description alone would substantially overestimate both the historical depth and the breadth of the modeling sample.

3.2 Data quality

Exploratory analysis (full notebook: `reports/R2_data_and_eda/eda.ipynb`) surfaced two further gaps between the data and its documentation, beyond the sample-size point above:

1. **Missing values are not NaN.** The preprocessing pipeline (`preprocess/eod.py`) pads missing trading days with a sentinel value of -1234 rather than a null. Undetected, this sentinel dominates naive summary statistics: raw skewness for every feature column comes out around -23 , which would be an extraordinary departure from normality for a bounded price ratio. Once the sentinel is stripped, skewness for every column falls to an unremarkable -0.3 to -0.4 . The original -23 number says nothing about the price data; it is an artifact of an undocumented missing-value code.
2. **The relation graph includes non-company tickers.** 156 of the 1,026 NASDAQ tickers (15.2%) are recorded with industry "n/a" in the sector/industry relation file. Inspecting these tickers shows they are ETFs and index funds (e.g. QQQ, SHV, IEF, PFF) rather than operating companies. An ETF's return is a basket average with no industry relation of its own, so its presence in a graph meant to encode company-to-company relations is noise rather than signal; 17 of these 156 tickers are fully isolated (degree zero) in the industry graph as a direct consequence.

Both issues are corrected identically across every model reported in Section 4: the -1234 sentinel is treated as missing (not as a value) wherever it appears, and results are reported with the 156 ETF/fund tickers still present in the universe unless noted otherwise, since removing them would change the size of the relation graph and is left as a robustness check in `reports/R3_model_and_experiments/PROGRESS.md`.

4 Methods

We train and compare two models, both sharing the same NASDAQ universe, the same 5-, 10-, 20-, 30-day moving-average and close-ratio features, and the same next-day ranking objective. Both are trained with the same optimizer (Adam), the same train/validation/test split by date (indices 0–755 / 756–1007 / 1008– of the 1,246 trading days), and the same combined loss: a mean-squared-error regression term on predicted return plus a pairwise ranking term that penalizes any pair of stocks whose predicted return order disagrees with their realized order, weighted by α .

4.1 Baseline: Rank_LSTM

The baseline (`training/rank_lstm.py` in the official repository) encodes each stock's own feature history with a single-layer LSTM and predicts next-day return from the final hidden state, with no information exchanged between stocks. This is the "stocks are independent" benchmark that Section 1 argues is the wrong assumption, and the lower bound RSR is designed to beat.

4.2 RSR: static relation graph

The RSR model (`training/relation_rank_lstm.py`) takes the same per-stock representation — here, the baseline LSTM's learned sequential embedding, exported after training rather than downloaded, since the original authors' pretrained embedding file is distributed only via an external link that we chose not to depend on — and propagates it across the static industry relation graph

via a learned, softmax-normalized attention weight over each stock’s neighbors, before combining the propagated and original representations into the final prediction. The relation graph itself, a $1,026 \times 1,026 \times 1$ one-hot encoding, is a training-time constant: identical on day one of training and day 1,246.

5 Experiments and Results

Both models are trained with $\text{seq} = 4$, $\text{unit} = 32$, the same Adam optimizer, and the same combined MSE-plus-ranking loss described in Section 4, on the same NASDAQ universe (1,026 tickers, Section 3), for the same 50-epoch schedule. We report three metrics on the held-out test split: *MSE*, the mean-squared error of predicted return; *mrrt*, the mean reciprocal rank of the true top-return stock (higher means the model’s top pick is more often the stock that actually gained the most); and *btl*, the simulated return of a strategy that buys the model’s top-ranked stock each day (a simple long-only backtest).

Table 1: Test-set performance, both models trained to 50 epochs on the same NASDAQ universe and embedding.

Model	Test MSE	Test mrrt	Test btl
Baseline (Rank_LSTM, no relation)	0.0003774	0.0488	1.018
RSR (static industry relation)	0.0003774	0.0276	1.130

Table 1 shows two results that both cut against the original paper’s headline claim.

First, **MSE is essentially identical across the two models** — the baseline and RSR agree to seven decimal places. Encoding the static industry relation graph does not lower mean-squared prediction error relative to treating stocks independently once training is run to convergence: RQ1, in the specific sense of “does a relation graph reduce MSE,” is not supported here.

Second, **the two ranking-adjacent metrics point in opposite directions**. The no-relation baseline has the higher *mrrt* (0.0488 vs. 0.0276), meaning it more often puts the single best-performing stock in first place; RSR has the higher simulated trading return (1.130 vs. 1.018), meaning a strategy that simply buys its top pick each day does better over the test period. Which model “wins” on RQ1 therefore depends on which of these two metrics one treats as the target: a strategy judged purely by whether it nails the single top pick favors the baseline, while a strategy judged by realized trading return over the test period favors RSR.

6 Discussion

6.1 Did the replication succeed?

Not in the direction Feng et al. (2019) report. The original paper’s central claim is that encoding relational information lowers prediction error and improves ranking relative to a relation-blind baseline; at convergence, our replication finds neither (Table 1, baseline vs. RSR rows). We see three non-exclusive reasons this replication may diverge from the original result, none of which we can rule in or out with a single training run:

1. **Hyperparameters were not tuned to this setting.** We used a single configuration ($\text{seq} = 4$, $\text{unit} = 32$) chosen for consistency between the baseline embedding and the relational model, not a value selected by the kind of hyperparameter search the original paper likely

performed. The original paper’s own example commands use larger sequence lengths and hidden sizes (e.g. `seq = 16`, `unit = 64` for their published wikidata run).

2. **A single training run.** Every number in Table 1 comes from one run per model with a fixed random seed. A single 50-epoch run is not by itself strong evidence that training has fully converged or that the ranking between models is stable; it is one draw, not a distribution.
3. **We used the pretrained embedding, but exported it ourselves.** Because the original authors’ pretrained sequential embedding is only available via an external link (Section 4), the embedding RSR consumes here was exported from our own baseline run rather than the authors’ released artifact, which may not be trained under the same conditions as the one used in the original paper’s reported numbers.

We do not have evidence to prefer one of these explanations over the others; distinguishing them is exactly what the robustness checks in `reports/R3_model_and_experiments/PROGRESS.md` (a second random seed; a brief hyperparameter sweep) are for, and that work is not yet done as of this draft.

6.2 Limitations

Results are reported for NASDAQ only, using the industry relation graph only (the original paper also reports a Wikidata-relation condition, which we did not run); the 156 ETF/fund tickers identified in Section 3 remain in the relation graph for both models reported here, rather than being excluded as their absence of a true industry relation would suggest; and only one ablation dimension (relation graph present vs. absent) was tested, on one market.

7 Conclusion and Future Work

This project replicated Feng et al. (2019)’s Relational Stock Ranking model on its official NASDAQ dataset, comparing it against a no-relation Rank_LSTM baseline under a matched training schedule (50 epochs, same optimizer, same embedding). At convergence, encoding the static industry relation graph did not lower mean-squared prediction error relative to the relation-blind baseline — the two models agree on MSE to seven decimal places — and the baseline in fact had the higher top-1 ranking accuracy (mrrt), while RSR had the higher simulated trading return (btl). The replication therefore does not reproduce the original paper’s headline claim that relational information improves both prediction error and ranking performance in this setting; the direction depends on which of the two ranking-adjacent metrics (mrrt vs. btl) one weights more.

Following the course’s framing of “potentials of your projects,” the most direct next steps are the remaining robustness checks scoped in `reports/R3_model_and_experiments/PROGRESS.md` — a second random seed and a brief hyperparameter sweep closer to the original paper’s published configuration — to establish whether the mrrt/btl split and the absence of an MSE improvement are stable findings or artifacts of a single run. Extending the comparison to the Wikidata relation and to NYSE, both available in the same official dataset but not run here, is the natural next step after that.

References

Cui, C., X. Li, C. Zhang, W. Guan, and M. Wang (2023). Temporal-relational hypergraph tri-attention networks for stock trend prediction. *Pattern Recognition*.

- Feng, F., X. He, X. Wang, C. Luo, Y. Liu, and T.-S. Chua (2019). Temporal relational ranking for stock prediction. *ACM Transactions on Information Systems* 37(2), 27.
- Jafari, A. and S. Haratizadeh (2022). Gcnet: Graph-based prediction of stock price movement using graph convolutional network. *arXiv preprint arXiv:2203.11091*.